A Distributed SQL Database Using MPI and SQLite: Design, Optimization, and Evaluation

Caleb Lefcort
TCSS 558 — Winter 2026
University of Washington Tacoma

***Abstract*—As datasets grow beyond what a single machine can store and process, distributed databases have become essential infrastructure for data-intensive applications. This paper presents the design, implementation, and evaluation of a distributed SQL database that partitions data across multiple processes. It uses MPI (Message Passing Interface) for inter-node communication and SQLite as the per-node storage engine. The system follows a coordinator-worker architecture with hash-based sharding, variable-length message serialization, pipelined batch processing, and transaction-aware write coalescing. We evaluate the system against single-node in-memory and file-based SQLite baselines across six SQL workload types (INSERT, targeted SELECT, full-scan SELECT, broadcast UPDATE, targeted DELETE, and mixed). Row counts range from 5,000 to 30,000 with 2–7 workers. After seven targeted optimizations, distributed INSERT throughput matches single-node file-based performance within 1% at 30,000 rows. Broadcast UPDATE latency converges to within 2% of single-file at 4 workers, showing that parallelism can offset communication overhead when per-operation computation is large enough.**

I. Introduction

Modern applications routinely generate and query datasets that exceed the capacity of a single server. Consider a company with 5 million customer records and 2 TB of data. Eventually, it will exhaust the storage, memory, and CPU of even the largest commercially available machine [1]. Vertical scaling, meaning purchasing faster hardware, provides diminishing returns as costs grow super-linearly. Physical limits on single-machine resources also impose hard ceilings on throughput and capacity. The alternative is horizontal scaling, which distributes both data and query processing across multiple machines. This allows systems to grow by adding commodity hardware rather than replacing it.

Distributed databases implement horizontal scaling by sharding (partitioning) data across nodes. They coordinate query execution so that clients observe a single logical database [2]. This coordination introduces communication overhead, consistency challenges, and failure-handling complexity that simply do not exist in single-node systems. Understanding these trade-offs is essential for anyone building and operating data systems at scale.

This paper describes the design, optimization, and benchmarking of a distributed SQL database built using MPI for communication and SQLite for storage. Our contributions are threefold. First, we present a coordinator-worker architecture with hash-based sharding, variable-length message serialization, and a two-phase SELECT result protocol that supports transparent query routing over MPI. Second, we describe seven performance optimizations, including blocking probes, pipelined batch sends, transaction wrapping, and variable-length messages. These reduced 1,000-row INSERT latency from 25,476 ms to 23 ms (a 1,108× speedup on the INSERT phase alone; 212× on end-to-end benchmark time). Third, we provide an empirical evaluation comparing distributed performance against single-node baselines. This evaluation covers six workload types, four row counts, and six worker counts, with analysis of when distributed parallelism offsets communication overhead.

II. Related Work

A. Production Distributed Databases

Google Spanner [2] pioneered globally distributed, strongly consistent SQL with TrueTime-based external consistency. CockroachDB [3] provides an open-source, serializable distributed SQL engine. It uses the Raft consensus protocol for replication and range-based sharding. Vitess [4] horizontally scales MySQL by interposing a proxy layer that routes queries to shards. It handles connection pooling, query rewriting, and schema management. TiDB [5] combines a distributed SQL layer with a transactional key-value store (TiKV), supporting both OLTP and OLAP workloads. These systems prioritize fault tolerance, multi-tenant concurrency, and geo-distribution, all of which are features beyond our scope. That said, they share the fundamental coordinator-shard architecture that our system implements.

B. MPI-Based Data Processing

MPI has traditionally been used for scientific computing rather than database workloads [6]. However, several projects have explored MPI for data-parallel SQL execution. Parallel database engines such as Gamma [7] and Volcano [8] demonstrated pipelined query execution across multiple nodes. Modern systems like Presto/Trino [9] and Apache Spark SQL [10] implement these concepts at scale using TCP-based communication. More recently, Widanage et al. [11] explored MPI-based DataFrame operations. They showed that MPI's low-latency message passing can outperform socket-based alternatives for tightly coupled parallel data processing. Our

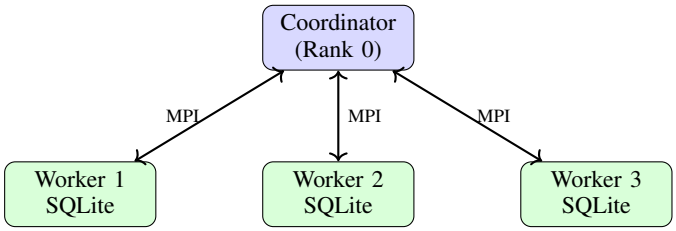

Fig. 1: Coordinator-worker architecture. The coordinator routes queries; each worker holds a shard in a local SQLite file.

system applies MPI in a similar spirit, but targets relational SQL workloads with a coordinator-worker model rather than a dataflow pipeline.

C. Embedded Database Engines

SQLite [12] is the most widely deployed database engine, designed for embedded, single-process use. Its simplicity makes it an attractive building block for distributed prototypes: it is a single library with file-based storage and full SQL support. LiteFS [13] and rqlite [14] extend SQLite with replication using FUSE and Raft respectively, but neither implements data sharding. Our work takes a complementary approach. Rather than replicating a single SQLite database, we shard data across multiple independent SQLite instances coordinated by MPI.

III. Methodology

A. System Architecture

The system follows a coordinator-worker model (Figure 1). A single coordinator (MPI rank 0) accepts SQL from the user or a batch file. It parses each statement, determines routing, dispatches commands to workers via MPI, and merges results. N workers (ranks $1 \ldots N{-}1$) each maintain an independent SQLite database file holding a shard of the data. Workers have no knowledge of each other. All communication flows through the coordinator. Each worker's SQLite instance uses the default rollback journal mode, matching the storage configuration of the single-node file-based baseline.

Every MPI process runs the same binary. At startup, `MPI_Comm_rank()` determines the rank. Rank 0 branches into the coordinator code path, and all other ranks enter the worker event loop. This single-binary design simplifies deployment and ensures all nodes run identical protocol logic.

B. Hash-Based Sharding

Data is partitioned across workers using hash sharding on the integer primary key:

$$\text{target_worker} = (\text{row_id} \bmod N_{\text{workers}}) + 1 \qquad (1)$$

where $N_{\text{workers}} = \text{world_size} - 1$ (excluding the coordinator). This function is deterministic, requires no lookup table, and distributes sequential keys uniformly. The $+1$ offset ensures workers map to MPI ranks $1 \ldots N{-}1$, reserving rank 0 for the coordinator.

Tag	Constant	Direction	Purpose
2	`TAG_CREATE_DB`	C→W	Create database file
3	`TAG_DROP_DB`	C→W	Delete database file
4	`TAG_USE_DB`	C→W	Switch active database
5	`TAG_TABLE_DDL`	C→W	DDL, BEGIN, COMMIT
6	`TAG_INSERT`	C→W	INSERT statement
7	`TAG_SELECT`	C→W	SELECT query
8	`TAG_UPDATE`	C→W	UPDATE statement
9	`TAG_DELETE`	C→W	DELETE statement
10	`TAG_RESULT`	W→C	Fixed-size result header
11	`TAG_ACK`	W→C	Integer ACK (0/1)
12	`TAG_RESULT_DATA`	W→C	Variable-length CSV
99	`TAG_EXIT`	C→W	Shutdown signal

TABLE I: MPI tag definitions

The command parser extracts shard keys using regular expressions. For INSERT statements, the parser matches the first value in the VALUES clause, which is assumed to be the primary key. For SELECT, UPDATE, and DELETE statements, it matches the row ID in a `WHERE id = n` clause. This is a simplifying assumption for our prototype: it requires that the primary key appear as the first column in INSERT statements and that targeted queries use a literal equality predicate on `id`. Queries that do not match these patterns (such as non-integer keys or alternate column orderings) fall through to the broadcast path, which is correct but slower. If no shard key is found, the query is broadcast to all workers. Otherwise, it is targeted to a single worker.

C. MPI Communication Protocol

1) Message Types: MPI tags distinguish message types on the communication channel. Table I lists all tags. Command messages from coordinator to worker are encapsulated in a `DatabaseMessage` struct containing the database name, SQL query, and a response flag.

2) Variable-Length Serialization: The `DatabaseMessage` struct contains fixed-size arrays (`db_name[256]`, `sql_query[4096]`), totaling 4,356 bytes. To avoid transmitting unused padding, we serialize to a compact wire format:

$$\underbrace{[\text{uint32}]}_{\text{db_len}}\underbrace{[\text{bytes}]}_{\text{db_name}}\underbrace{[\text{uint32}]}_{\text{sql_len}}\underbrace{[\text{bytes}]}_{\text{sql}}\underbrace{[\text{int32}]}_{\text{flag}}$$

A `BEGIN` command serializes to ∼20 bytes instead of 4,356 bytes (a 218× reduction). On the receiving end, `MPI_Probe` followed by `MPI_Get_count` discovers the incoming message size. The receiver then allocates a buffer and calls `MPI_Recv`.

3) Write Protocol: For write operations (INSERT, UPDATE, DELETE), each worker executes the SQL via `sqlite3_exec()` and sends back a single integer ACK on `TAG_ACK`. A value of 0 indicates success, and 1 indicates an error. On error, the coordinator logs the failing query and the worker's error message to standard error, then continues

Algorithm 1 Query Routing

Require: Parsed command c with type, shard key k, SQL text

```
if c.type ∈ {DDL, BEGIN, COMMIT} then
    Broadcast to all workers via TAG_TABLE_DDL
    Collect N ACKs
else if k ≠ −1 then
    w ← (k mod N) + 1 {Targeted}
    Send to worker w; collect 1 ACK or result
else
    Broadcast to all workers {No shard key}
    Collect N ACKs or results; merge if SELECT
end if
```

processing subsequent commands. No retry or rollback is attempted; the system treats individual statement failures as non-fatal, consistent with SQLite's own behavior in autocommit mode.

4) Read Protocol: SELECT queries use a two-phase response protocol. In Phase 1, the worker sends a fixed-size result header on `TAG_RESULT` containing the worker rank, row count, and status. In Phase 2, if rows were found, the worker sends a variable-length CSV string on `TAG_RESULT_DATA`. For broadcast SELECTs, the coordinator collects all Phase 1 headers using `MPI_ANY_SOURCE`, accepting responses in whatever order they arrive. It then collects Phase 2 data payloads from workers that reported rows.

D. Query Routing

Algorithm 1 summarizes the coordinator's routing logic. DDL commands (CREATE/DROP TABLE) are always broadcast to maintain schema consistency. Data operations are targeted when a shard key is present and broadcast otherwise.

E. Performance Optimizations

The unoptimized system exhibited a 25,000× overhead for 1,000 INSERTs compared to single-node SQLite. We traced this to three root causes and applied seven optimizations.

1) Optimization 1: Blocking MPI_Probe: Workers originally used a non-blocking `MPI_Iprobe` with a 10 ms sleep between polls. This added roughly 5 ms of latency per message on average. Replacing the polling loop with a blocking `MPI_Probe` brought per-operation latency down from ∼20 ms to ∼1–2 ms.

2) Optimization 2: Pipelined Batch Sends: The coordinator originally sent each INSERT and waited for the ACK before sending the next. This left $N-1$ workers sitting idle. We implemented look-ahead batch detection, where the file executor scans consecutive statements of the same type and dispatches all messages before collecting any ACKs. This way, all workers can process their share concurrently.

3) Optimization 3: Transaction Wrapping: In SQLite's autocommit mode, each write triggers an `fsync()` that costs roughly 6 ms. With 333 inserts per worker, this alone adds up to ∼2,000 ms. Wrapping each batch in explicit `BEGIN`/`COMMIT` reduces fsyncs from $N_{\text{rows_per_worker}}$ to exactly 1 per worker, regardless of batch size.

Operation	Messages Sent	Messages Received
Targeted write	1	1 (ACK)
Broadcast write	N	N (ACKs)
Targeted SELECT	1	2 (header + data)
Broadcast SELECT	N	$\leq 2N$ (headers + data)
Batch of B writes	$B + 2N$	$B + 2N$

TABLE II: Communication complexity per operation (N = number of workers)

4) Optimizations 4–5: Batch DELETE and UPDATE: We applied the same pipelined-batch-plus-transaction pattern to consecutive DELETEs and UPDATEs. For broadcast UPDATEs, the batch handler tracks expected ACKs as batch_size × N_{workers}.

5) Optimization 6: Pipelined SELECT Collection: SELECT result collection was restructured into two phases. All fixed-size headers are received first using `MPI_ANY_SOURCE`, and then all variable-length data payloads follow. This prevents one slow worker from blocking receipt of other workers' headers.

6) Optimization 7: Variable-Length Messages: As described in Section III-C2, serialization reduces per-message size from 4,356 bytes to 20–80 bytes. For 1,000 INSERTs, total MPI traffic drops from 4.3 MB to ∼80 KB.

7) Communication Complexity: Table II summarizes the message complexity per operation.

For a batch of B writes, the $2N$ additional messages account for the `BEGIN` and `COMMIT` broadcasts. The total communication cost is $O(B + N)$ rather than the unbatched $O(B \cdot N)$ for broadcast operations.

IV. Evaluation

A. Experimental Setup

All experiments were conducted on a single machine running Windows 10 (WSL2) with an 8-core CPU and 16 GB RAM. MPI processes communicate via shared memory, with no network involved. This setup creates a mixed tradeoff compared to a true multi-machine cluster. On the one hand, communication is faster than it would be over a real network, since shared-memory message passing avoids network latency entirely. On the other hand, all processes contend for the same CPU cores, memory bus, and physical disk, so the system cannot realize true hardware parallelism. The net effect is that our benchmarks understate communication overhead (which would be higher on a real network) but also understate parallelism gains (which would be higher with independent hardware). On a true cluster, the crossover point where distributed outperforms single-node would likely shift to smaller datasets.

B. Baselines

Three implementations were compared. The first is a single-node in-memory baseline using plain SQLite with an in-

Configuration	INSERT (ms)	Total (ms)	Speedup
Baseline (unoptimized)	25,476	25,476	1×
+ Blocking probe	4,189	4,405	6×
+ Pipelined sends	4,189	4,405	6×
+ Transaction wrapping	**23**	**120**	**212**×

TABLE III: Cumulative optimization impact (1,000 INSERTs, 3 workers)

memory database and no disk I/O. This represents the theoretical minimum SQL execution time. The second is a single-node file-based baseline using SQLite writing to a `.db` file with the default rollback journal and real fsyncs. This matches the storage pattern of the distributed workers. The third is our distributed MPI-based system with 1 coordinator + N workers, each running file-based SQLite. The file-based benchmark applies the same BEGIN/COMMIT transaction wrapping as the distributed system to ensure a fair comparison.

C. Workloads

Six workload types were tested. INSERT batch inserts n rows with sequential integer IDs. SELECT targeted runs 50 point lookups by primary key, while SELECT full scan performs 5 full table scans. UPDATE broadcast executes 10 updates on a non-shard-key column, requiring broadcast to all workers. DELETE targeted removes 50 rows by primary key. Finally, a mixed workload runs all operation types in sequence: n INSERTs, 3 targeted SELECTs, 1 full table scan, 1 broadcast UPDATE, 3 targeted DELETEs, and a final full scan. The mix is INSERT-heavy by design, reflecting a write-dominated OLTP pattern.

Row counts ranged from 5,000 to 30,000, and worker counts from 2 to 7. Each configuration was run 3 times, and we report the mean.

D. Results

1) Optimization Impact: Table III shows the cumulative impact of the three primary optimizations on 1,000-row INSERT latency. Figure 2 visualizes this progression on a log scale. Transaction wrapping dominated because it eliminated ∼333 fsyncs per worker (∼6 ms each), reducing disk wait from ∼2,000 ms to ∼6 ms per worker. Pipelined sends showed minimal standalone impact because fsync cost still dominated. Their benefit only materialized after fsync elimination enabled true parallelism.

2) Cross-Implementation Comparison at 30,000 Rows: Figure 3 and Table IV present the final optimized results at 30,000 rows.

INSERT is the headline result: 260 ms distributed vs. 258 ms single-file, within 1%. The per-row SQL work (parsing, B-tree insertion, journal management) dominates both systems equally. This makes MPI overhead a negligible fraction of the total.

SELECT full scan shows the distributed system (346 ms) outperforming single-file (374 ms) by 7%. All workers scan

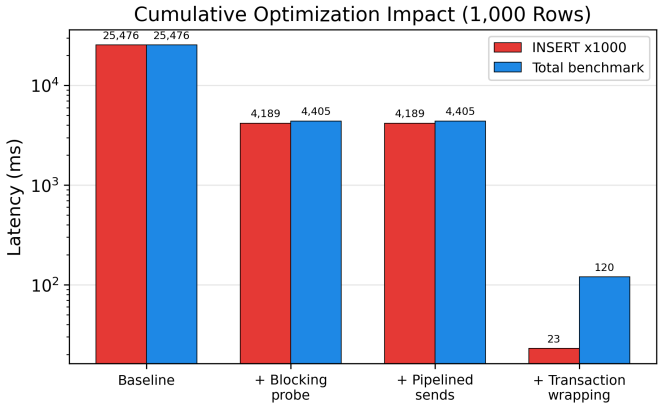

Fig. 2: Cumulative optimization impact on 1,000-row workload (log scale). Transaction wrapping yields the dominant improvement, reducing INSERT latency from 4,189 ms to 23 ms.

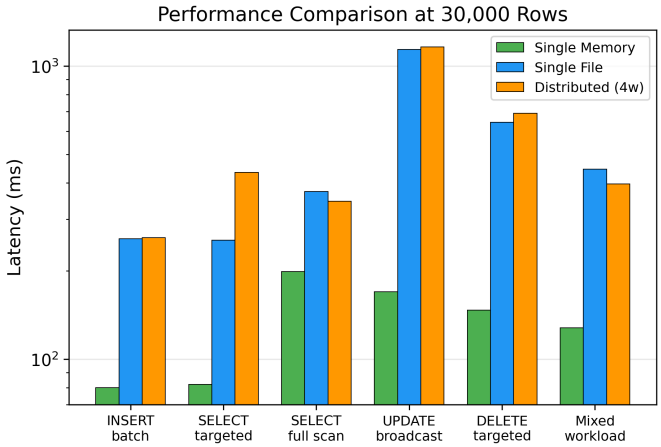

Fig. 3: Performance comparison across all three implementations at 30,000 rows (log scale). INSERT and UPDATE show near-parity between distributed and single-file, while targeted operations expose MPI overhead.

their shard in parallel, and the reduced per-worker data size outweighs the fan-in cost.

UPDATE broadcast shows near-parity: 1,163 ms distributed vs. 1,141 ms single-file (within 2%). Each of the 10 UPDATEs scans all matching rows. With 4 workers, each scans ∼7,500 rows instead of 30,000, and they do so in parallel.

DELETE targeted and SELECT targeted both show distributed overhead (690 vs. 644 ms and 434 vs. 255 ms respectively). The per-operation SQL work is trivial since it is just a microsecond-scale primary key lookup. As a result, the 1–2 ms MPI round-trip per operation dominates the total time.

3) Worker Scaling: Figure 4 shows how distributed performance scales with worker count at 30,000 rows.

INSERT scales well, going from 336 ms at 2 workers down to 239 ms at 7 workers (1.4×). This is because each worker handles fewer B-tree insertions as the data gets spread out more. Full scan and UPDATE plateau at 4–5 workers and show

Operation	Memory	File	Dist (4w)
INSERT batch	80	258	260
SELECT targeted (×50)	82	255	434
SELECT full scan (×5)	199	374	346
UPDATE broadcast (×10)	170	1,141	1,163
DELETE targeted (×50)	147	644	690
Mixed workload	128	445	396

TABLE IV: Benchmark results at 30,000 rows (mean of 3 runs, ms)

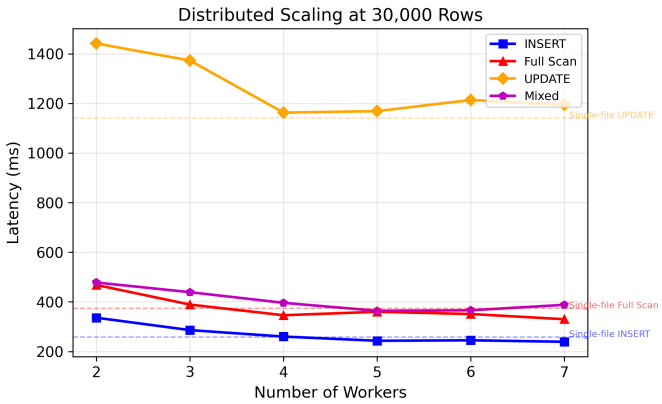

Fig. 4: Distributed latency vs. worker count at 30,000 rows. Dashed lines indicate single-file baselines. INSERT and mixed workloads improve monotonically up to 5 workers. UPDATE and full scan plateau at 4–5 workers due to shared-resource contention.

no further improvement beyond that point. Several factors explain this ceiling. First, the coordinator is a serial bottleneck. It must dispatch all commands and collect all responses, so adding workers increases the fan-in/fan-out work on rank 0 even as per-worker computation shrinks. Second, all processes run on the same 8-core machine. Beyond 4–5 workers (plus the coordinator), the system is oversubscribed and processes compete for CPU time rather than running in true parallel. Third, disk contention grows with worker count since all workers fsync to the same physical drive, and the OS must serialize those writes. The net effect is that the per-worker computation savings from additional sharding are offset by increased coordination and resource contention, producing diminishing returns.

4) Row Count Scaling: Figure 5 compares all three implementations as row count increases for the INSERT and UPDATE workloads. As row count increases, the per-row SQL computation grows proportionally while MPI overhead remains roughly constant per operation. This causes the distributed-to-single-file ratio to approach 1.

5) Message Size Impact: Variable-length serialization reduced per-message size by 54–218× (Table V). For 1,000 INSERTs, total MPI traffic dropped from 4.3 MB to ∼80 KB. On shared-memory MPI, this reduces `memcpy` overhead. On a real network, the savings would be even more significant since

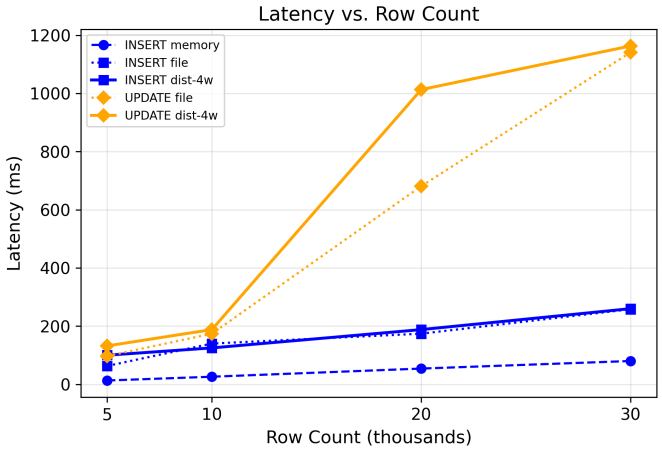

Fig. 5: Latency vs. row count. INSERT distributed converges with single-file as row count grows. UPDATE distributed tracks single-file closely at all scales.

Query Type	Fixed (B)	Serialized (B)	Reduction
`BEGIN/COMMIT`	4,356	∼20	218×
`DELETE ... WHERE`	4,356	∼55	79×
`INSERT ... VALUES`	4,356	∼80	54×

TABLE V: Message size before and after variable-length serialization

smaller messages reduce bandwidth consumption and network queuing delays.

E. Discussion

1) When Does Distribution Pay Off?: Our results show that distributed performance matches single-file when per-operation computation is large relative to communication cost. INSERT (parsing + B-tree insertion per row) and broadcast UPDATE (scanning thousands of rows) are computation-heavy. In these cases, MPI overhead gets amortized over the heavier SQL work. Targeted DELETE and SELECT involve trivial SQL work (a single primary key lookup), so the 1–2 ms MPI round-trip ends up dominating.

2) Single-Machine Limitations: All experiments ran on one machine. The distributed system pays three costs that would not exist on a true cluster. First, CPU contention: all MPI processes compete for the same cores and L3 cache. Second, disk contention: multiple workers fsync to the same physical disk. Third, there is no true parallelism gain because shared-memory MPI provides message passing semantics but not independent hardware resources. At the same time, shared-memory communication is faster than a real network, meaning our benchmarks also understate the communication overhead that would exist on a true cluster. The net effect is that our results compress both sides of the tradeoff: parallelism gains are understated, but so is communication cost. On separate machines with independent CPUs, memory buses, and disks, the parallelism benefits would be realized fully, though network latency would add per-message cost that shared memory

avoids. The crossover point where distributed outperforms single-node would likely shift to smaller datasets overall.

V. Conclusion

We designed, implemented, and evaluated a distributed SQL database using MPI for inter-process communication and SQLite as the per-node storage engine. Through seven targeted optimizations (blocking probes, pipelined batch sends, transaction wrapping, and variable-length messages), we reduced distributed overhead from 25,000× to near-parity with single-node for computation-heavy operations at 30,000 rows. The key insight is that batching, and specifically eliminating per-row fsync overhead via explicit transactions, is the single most impactful optimization in any database system, distributed or not. Our results confirm the fundamental distributed systems trade-off. Communication overhead is inherent and unavoidable. But when per-operation computation exceeds communication cost, parallelism across shards delivers real throughput gains.

Future work could extend the system in several directions. Data replication using Raft consensus would add fault tolerance. Distributed aggregation pushdown, where SUM, COUNT, and AVG are computed per shard and merged at the coordinator, would reduce data movement. Multi-machine deployment would allow evaluation of true network-separated parallelism. Finally, concurrent client support via coordinator thread pooling would improve real-world usability.

The source code is available at https://github.com/CalebLefcort/Distributed_SQLite_DB.

References

[1] M. T. Özsu and P. Valduriez, *Principles of Distributed Database Systems*, 4th ed. Springer, 2020.

[2] J. C. Corbett *et al.*, "Spanner: Google's globally distributed database," *ACM Trans. Comput. Syst.*, vol. 31, no. 3, pp. 1–22, Aug. 2013.

[3] R. Taft *et al.*, "CockroachDB: The resilient geo-distributed SQL database," in *Proc. ACM SIGMOD*, 2020, pp. 1493–1509.

[4] S. Sougoumarane, "Vitess: Scaling MySQL at YouTube using Go," presented at USENIX LISA, San Diego, CA, 2012.

[5] D. Huang *et al.*, "TiDB: A Raft-based HTAP database," *Proc. VLDB Endow.*, vol. 13, no. 12, pp. 3072–3084, 2020.

[6] W. Gropp, E. Lusk, and A. Skjellum, *Using MPI: Portable Parallel Programming with the Message-Passing Interface*, 2nd ed. MIT Press, 1999.

[7] D. J. DeWitt *et al.*, "The Gamma database machine project," *IEEE Trans. Knowl. Data Eng.*, vol. 2, no. 1, pp. 44–62, Mar. 1990.

[8] G. Graefe, "Volcano—an extensible and parallel query evaluation system," *IEEE Trans. Knowl. Data Eng.*, vol. 6, no. 1, pp. 120–135, Feb. 1994.

[9] R. Sethi *et al.*, "Presto: SQL on everything," in *Proc. IEEE ICDE*, 2019, pp. 1802–1813.

[10] M. Armbrust *et al.*, "Spark SQL: Relational data processing in Spark," in *Proc. ACM SIGMOD*, 2015, pp. 1383–1394.

[11] C. Widanage *et al.*, "High performance data engineering everywhere," in *Proc. IEEE Int. Conf. Smart Data Services (SMDS)*, 2020, pp. 122–132.

[12] D. R. Hipp, "SQLite," 2000. [Online]. Available: https://www.sqlite.org/

[13] Fly.io, "LiteFS: Distributed SQLite," 2023. [Online]. Available: https://github.com/superfly/litefs

[14] P. O'Toole, "rqlite: The lightweight, distributed relational database built on SQLite," 2023. [Online]. Available: https://github.com/rqlite/rqlite